OBJECT DESIGN

Favourite Books Online Store

WILLIAM REEDIE, NGY KEANG OEUNG, VAN TUY, ANH PHAN
104300597, 104201881, 104482107, 104480541

Executive Summary

Favourite Books is a bookstore located on Glenferrie Road that is expanding its operations into an online environment to improve accessibility and support business growth. The proposed online bookstore system is intended to support online browsing, purchasing, payment processing, shipment tracking, and catalogue management through a fully electronic system.

This document presents the initial object-oriented design for the proposed solution. The report includes candidate classes and their relationships, CRC cards, design heuristics and patterns, the bootstrap process, and several use scenarios used to verify the proposed design.

Contents

1. Introduction and Overview

This Object Design Document describes the initial object-oriented design developed for the online bookstore system. The design focuses on identifying the system classes, their responsibilities, relationships, and interactions within the online bookstore system.

2. Problem Analysis

2.1 Goals

The main goals of the online bookstore system are:

- Allow customers to browse and search books online
- Allow customers to create accounts and securely log into the system
- Support adding and managing books in a shopping cart
- Enable online ordering and payment processing
- Generate invoices and transaction records
- Provide shipment tracking for customer orders
- Manage inventory and stock availability
- Support administrative tasks such as catalogue updates and sales reporting
- Improve accessibility and operational efficiency through a fully electronic system

2.2 Assumptions

A1 All books stored in the system have a unique book identifier.

A2 Customers must register an account before placing an order.

A3 Customers can browse the catalogue without logging into the system.

A4 The system supports online payment processing through external payment services.

A5 All payments must be successfully verified before an order is confirmed.

A6 The bookstore only sells physical books.

A7 Each order belongs to exactly one customer.

A8 An order may contain multiple books.

A9 Inventory levels are updated automatically after successful purchases.

A10 Shipment tracking information becomes available after an order is dispatched.

A11 Administrative users are responsible for managing book listings, categories, and stock information.

A12 Sales reports are generated using stored order and transaction data.

A13 Customers can access their order history and shipment status through their accounts.

A14 The database system operates in a secure environment.

A15 Internet access is required for customers to use the online bookstore system.

3. Candidate Classes

3.1 List of Candidate Classes

- User
  - Customer
  - Admin
- Book
- Catalogue
- Cart
- Order
  - OrderItem
- Payment
  - Payment Method
    - Card
    - OnlinePayment
- Invoice
- Shipment
- Category
- Inventory
- SalesReport
- Database
- PaymentService

3.2 CRC Cards

3.2.1 User

Class Name: User **Super Class** -	
A cart temporarily stores selected books before checkout.	
Responsibility	**Collaborator**
Knows user details	Book
Can authenticate login	Database

3.2.2 Customer

Class Name: Customer **Super Class: User**	
A customer represents a registered user of the system who can browse books, place orders, and track purchases.	
Responsibility	**Collaborator**
Browses catalogue	Catalogue
Searches/filters books	Catalogue
Adds books to cart	Cart
Modifies cart items	Cart
Places an order	Order
Provides delivery details	Order
Initiates payment	Payment
Views order history	Order
Tracks shipment status	Shipment

3.2.2 Book

Class Name: Book **Super Class** -	
A book represents a product available for sale in the system.	
Responsibility	**Collaborator**
Stores book details	
Tracks stock availability	Inventory
Classifies category	Category
Stores book information	Catalogue

3.2.3 Catalogue

Class Name: Catalogue **Super Class** -	
The catalogue represents a collection of books available for browsing and searching.	
Responsibility	**Collaborator**
Stores collection of books	Book
Displays available books	Book
Searches books	Category
Updates book listing	Admin

3.2.4 Cart

Class Name: Cart **Super Class** -	
A cart temporarily stores selected books before checkout.	
Responsibility	**Collaborator**
Can add book to cart	Book
Can remove book from cart	Book

Can update quantity	Book
Can calculate total price	Book

3.2.5 Order

Class Name: Order **Super Class** -	
An order represents a confirmed purchase made by a customer.	
Responsibility	**Collaborator**
Knows customer placing order	Customer
Stores order items	OrderItem
Calculates total cost	Cart
Links payment	Payment
Generates invoice	Invoice
Tracks order status	Shipment

3.2.6 OrderItem

Class Name: OrderItem **Super Class** -	
An order item represents a specific book within an order.	
Responsibility	**Collaborator**
Knows selected book	Book
Knows quantity	
Calculates subtotal	Book

3.2.7 Payment

Class Name: Payment **Super Class** -	
The payment class handles transaction processing.	
Responsibility	**Collaborator**
Receives payment details	Customer
Processes payment	PaymentService
Verifies transaction	PaymentService
Records payment	Database
Confirms payment success	Order

3.2.8 PaymentMethod

Class Name: PaymentMethod **Super Class** -	
PaymentMethod represents an abstract way of handling different types of payment. It defines the general behavior that all payment types must follow.	
Responsibility	**Collaborator**
Defines payment behaviour	Payment
Provides interface for payment processing	Payment

3.2.9 Card

Class Name: Card **Super Class: PaymentMethod**	
Card represents a payment method using credit or debit card details to complete a transaction.	
Responsibility	**Collaborator**
card details	
Processes card payment	PaymentService

3.2.10 OnlinePayment

Class Name: OnlinePayment **Super Class:** PaymentMethod	
OnlinePayment represents a digital payment method that uses an external online service to complete transactions.	
Responsibility	**Collaborator**
Handles online payment process	PaymentService
Verifies transaction result	PaymentService

3.2.11 PaymentService

Class Name: PaymentService **Super Class -**	
PaymentService represents an external system responsible for processing and confirming payment transactions.	
Responsibility	**Collaborator**
Processes payment request	Payment
Returns transaction result	Payment

3.2.12 Invoice

Class Name: Invoice **Super Class** -	
An invoice records completed transactions.	
Responsibility	**Collaborator**
Generates invoice	Order
Generates receipt	Payment
Stores transaction record	Database
Sends receipt to customer	Customer

3.2.13 Shipment

Class Name: Shipment **Super Class** -	
Shipment handles delivery and tracking.	
Responsibility	**Collaborator**
Creates shipment record	Order
Assigns delivery details	Customer
Updates shipment status	Order
Provides tracking information	Customer

3.2.14 Inventory

Class Name: Inventory **Super Class** -	
Inventory manages stock levels.	
Responsibility	**Collaborator**
Tracks stock levels	Book
Updates stock after purchase	Order
Validates stock availability	Cart

3.2.15 Category

Class Name: Category **Super Class** -	
Category organizes books.	
Responsibility	**Collaborator**
Classifies books	Book
Supports filtering	Catalogue

3.2.16 SalesReport

Class Name: SalesReport **Super Class** -	
SalesReport provides analytics.	
Responsibility	**Collaborator**
Retrieves sales data	Order
Generates report	Admin
Identifies best selling books	Book

3.2.17 Admin

Class Name: Admin **Super Class: User**	
Admin manages system operations.	
Responsibility	**Collaborator**
Manages book listings	Book
Updates catalogue	Catalogue
Manages categories	Category
Monitors stock	Inventory
Views sales reports	SalesReport

3.2.18 Database

Class Name: Database **Super Class** -	
The database stores system data and executes queries.	
Responsibility	**Collaborator**
Knows connection details	
Executes queries	

3.3 Justification

The candidate classes for the online bookstore system were identified through analysis of the domain entities, tasks, and workflows defined in Assignment 1. Core classes such as Customer, Book, Catalogue, Cart, Order, Payment, Invoice, and Shipment were selected because they directly represent real world objects and processes involved in an online purchasing system. For example, the Customer class captures user interactions such as browsing, ordering, and tracking deliveries, while the Order class represents the central transaction that links customers, payments, and shipments.

Supporting classes including OrderItem, Category, Inventory, and SalesReport were introduced to improve separation of concerns and ensure that responsibilities are distributed appropriately. For instance, OrderItem allows individual books and quantities to be managed separately within an order, while Inventory is responsible for stock management rather than overloading the Book class. Similarly, Category enables efficient classification and filtering of books, and SalesReport supports administrative analysis without affecting operational classes.

The Admin class was included to represent system management responsibilities such as updating catalogue data, monitoring stock, and generating reports, reflecting the administrative tasks outlined in the system requirements. The Database and PaymentService were considered as supporting or external classes to represent data persistence and third-party payment processing, as described in the system assumptions.

Certain potential classes were deliberately excluded or simplified. For example, user interface elements such as forms or controllers were not included, as they belong to implementation-level design rather than object-oriented domain modelling. Additionally, Invoice and Receipt were combined into a single Invoice class to reduce redundancy, as both serve similar purposes in recording transaction details.

Overall, the selected candidate classes reflect a modular and well structured design that aligns with the system requirements, avoids the creation of overly complex “god classes,” and ensures that each class has a clear and focused responsibility.

4. Design Patterns

4.1 Patterns Summary

4.1.1 Model-View-Controller Pattern

MVC is used as the overall architectural direction to separate the user interface from the business logic, particularly as interface changes are likely throughout development. The Model layer consists of the core bookstore entities defined in this design, while the View and Controller layers are not included at this stage of the object design.

4.1.2 Strategy Pattern

Payment processing required a design that would not depend entirely on one payment implementation, especially since the requirements already assume integration with external payment services. Different payment methods, such as card payments or third-party services, would affect the payment flow differently. Due to this, a strategy-based approach was considered for the payment component. A common PaymentStrategy interface could separate different payment implementations instead of placing all payment logic into one class.

4.1.3 Factory Method Pattern

Different user roles already exist within the requirements specification, including Customer, Admin. Since these objects share some similar account-related behaviour while still representing different system roles, object creation was one area where a Factory Method approach appeared suitable.

4.1.4 Observer Pattern

Order and stock updates involve multiple parts of the system reacting after payment has been completed. For example, stock quantities need updating, invoices need generating, and the order status also changes once the transaction succeeds. Because several components respond to the same event, an Observer-style approach seemed appropriate for handling these updates while keeping the components separated from each other.

4.2 Heuristics

H1: A class should capture one and only one key abstraction
Different classes are separated and each has its own responsibility, meaning that they are not responsible to handle multiple things at once

H2: Keep related data and behaviour together
The design keeps the behaviour together with the data it belongs to

H3: Distribute system intelligence evenly
The design avoids putting all system functionality into one large class. Features such as account management, catalogue management, payment processing, shipment tracking, and sales reporting are separated into different classes.

H4: Maintain consistency in naming and domain terminology
The design keeps the terminology like Customer, Book, Cart etc consistent with the requirements specification and domain model throughout the system.

H5: Use inheritance only for specialisation relationships

Inheritance is only used where there is a clear relationship between the classes such as Administrator and Customer. These are specialised types of User because they share common behaviours such as storing user details and authentication. They then extend the User class with their own responsibilities, where Customer handles browsing and ordering functions, while Admin manages catalogue, stock, and reporting tasks. Other relationships in the system use associations instead of inheritance

4.3 Bootstrap Process

The bootstrap process describes how the online bookstore system is initialised and how the main objects are created and connected when the application begins execution. The process ensures that the system components are loaded in the correct order before customers can interact with the bookstore services.

The online bookstore system follows a layered initialisation process where the database connection and catalogue information are loaded first, followed by user interaction components and transaction related objects.

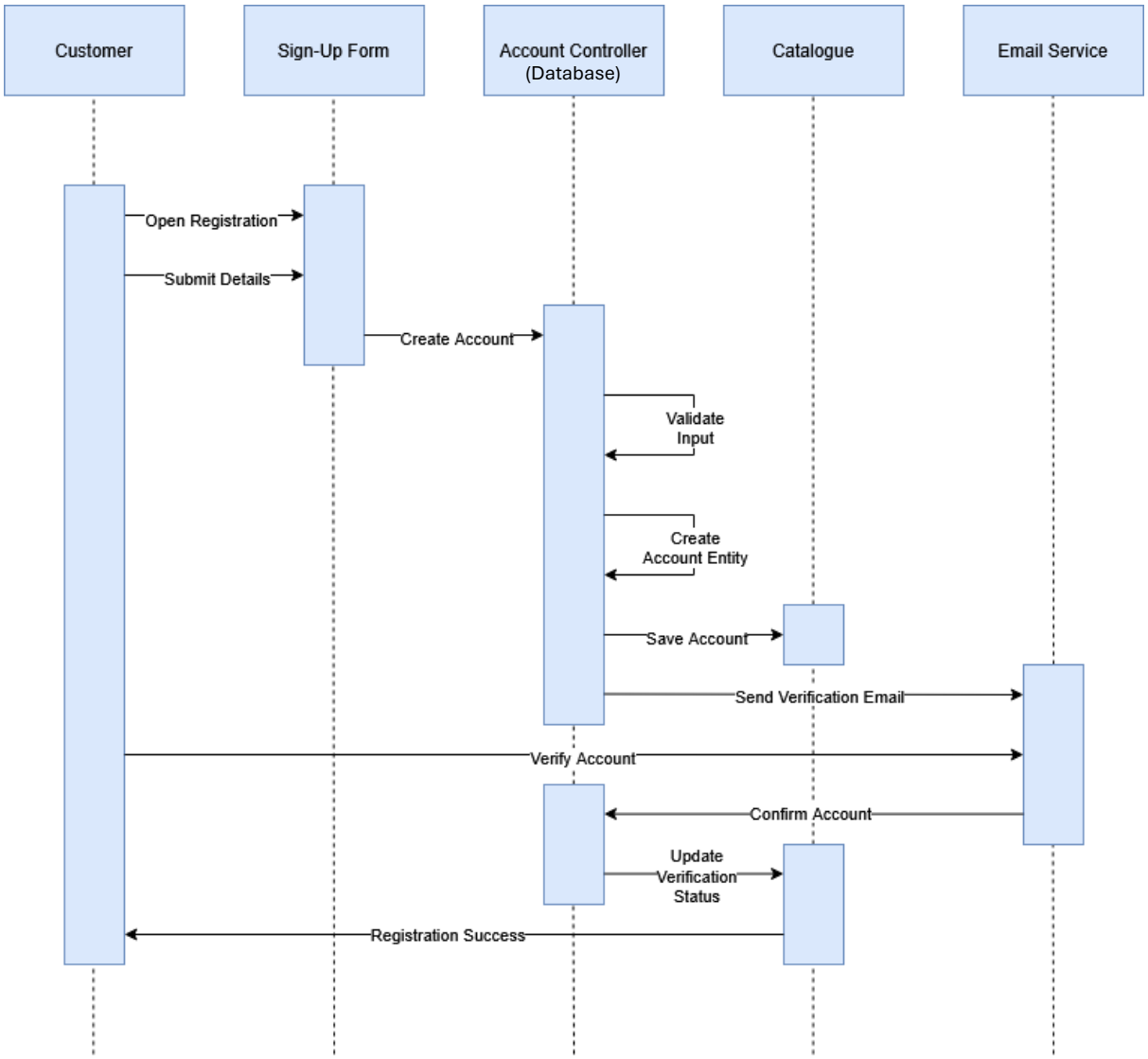
The bootstrap process for a customer purchasing books is described below:

- Main creates an instance of the Database class
- Database establishes the system connection and loads stored data
- Main creates an instance of the Catalogue class
- Catalogue retrieves available book information from the Database
- Catalogue creates Book objects for all available books
- Main creates the Customer object after the user registers or logs into the system
- Customer creates a Cart object when books are added to the cart
- Cart stores selected Book objects and their quantities
- Customer initiates checkout and creates an Order object
- Order creates multiple OrderItem objects based on the selected books
- Order calculates the total cost of the purchase
- Order creates a Payment object for transaction processing
- Payment communicates with the PaymentService to verify and process payment
- After successful payment, the Order creates an Invoice object
- Invoice stores transaction information in the Database
- Order creates a Shipment object to manage delivery and tracking information
- Inventory updates stock levels after the order is confirmed
- Shipment provides tracking updates to the Customer

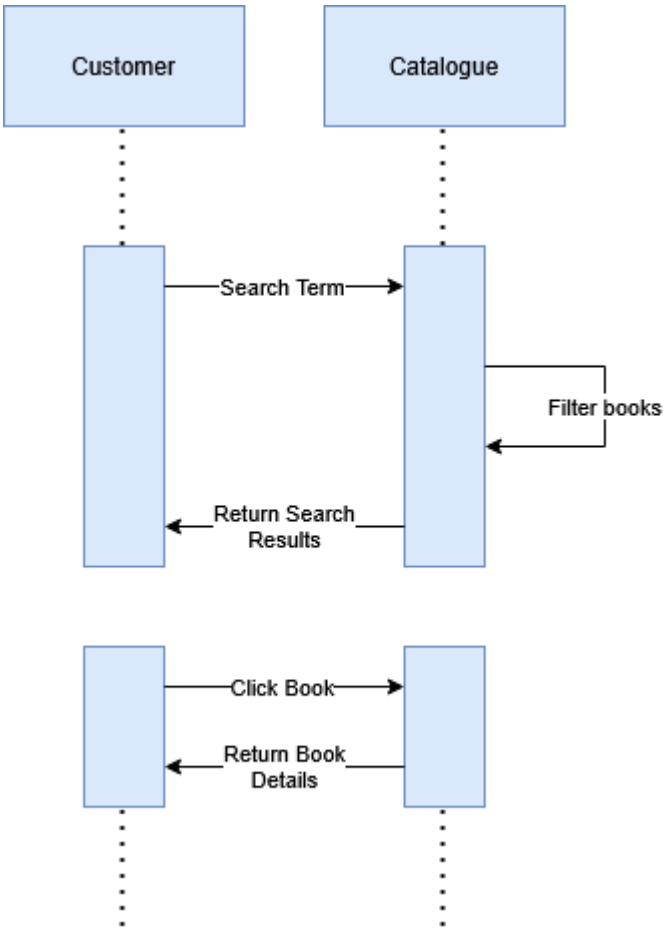
This bootstrap process ensures that responsibilities are distributed across specialised classes rather than concentrating functionality into a single class. The process also supports scalability because additional services such as recommendation systems, loyalty programs, or external delivery APIs can be integrated into the workflow without significantly affecting the existing object structure.
The bootstrap process also demonstrates how the system follows object-oriented principles such as separation of concerns and modularity by assigning each class a specific responsibility during system initialisation and transaction processing.

5. Verification

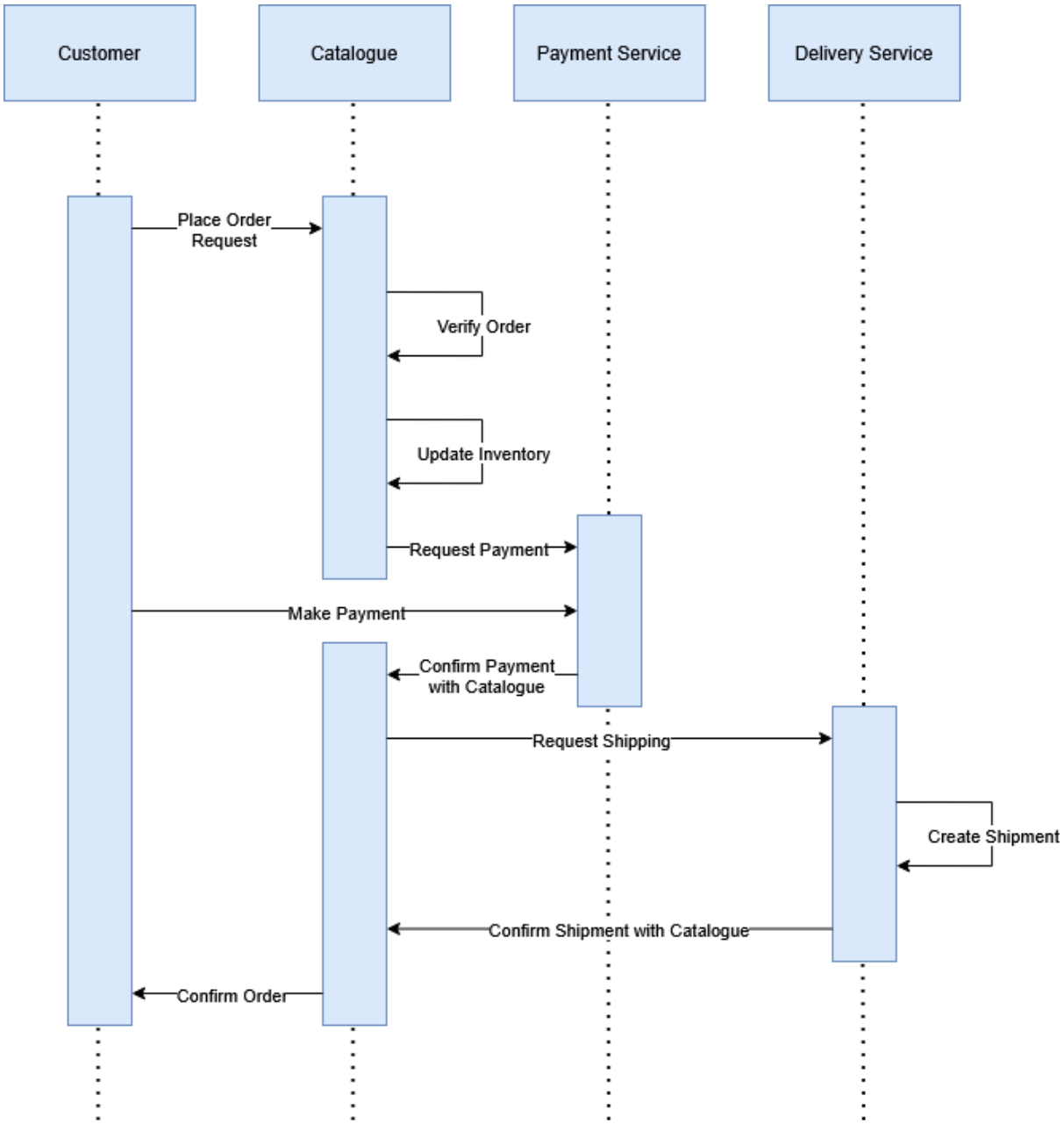
5.1 Sequence Diagram 1 – Create Customer Account



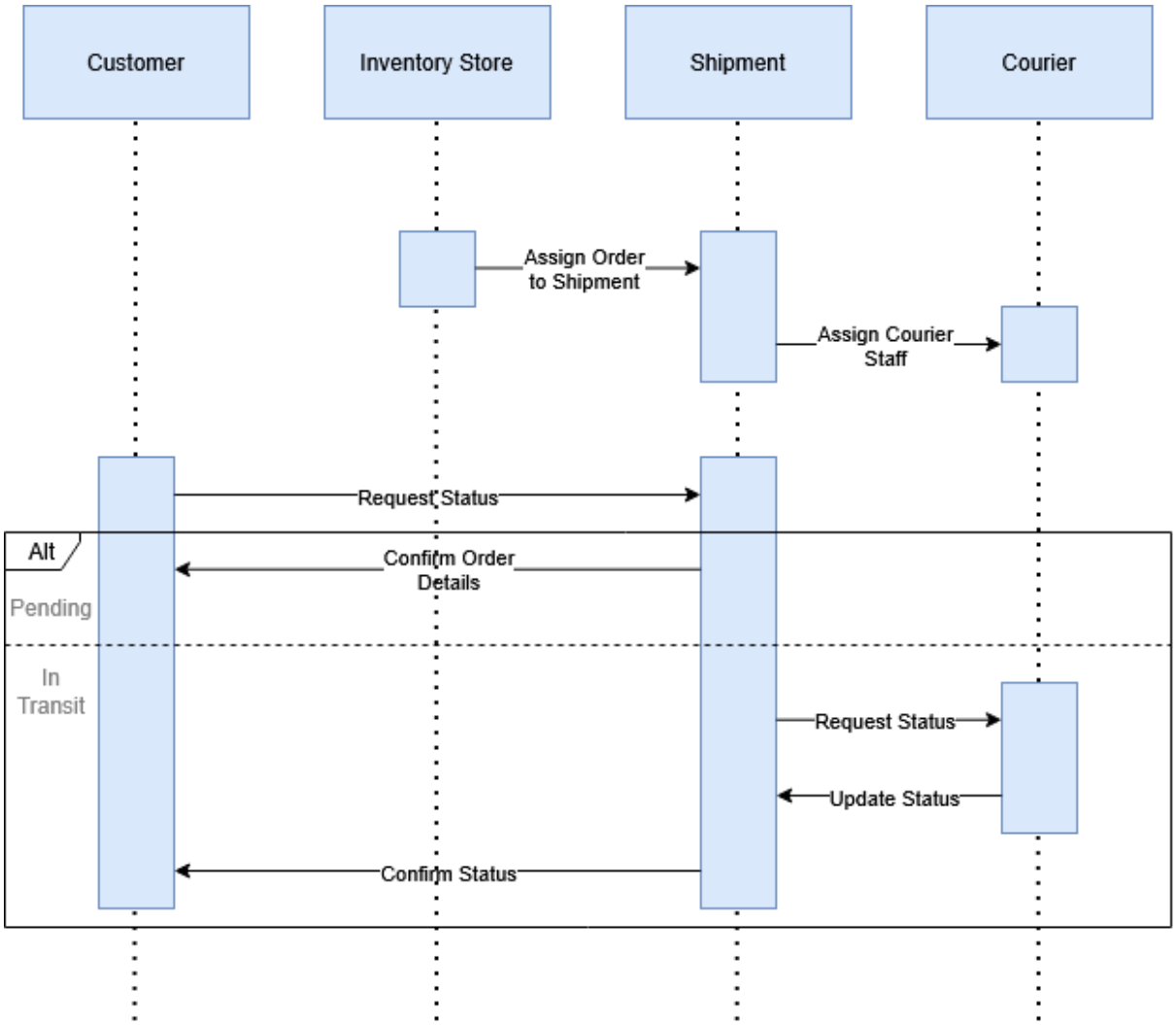
5.2 Sequence Diagram 2 – Search and Browse Books



5.3 Sequence Diagram 3 – Complete Payment Process



5.4 Sequence Diagram 4 – Order Delivery



6. Appendix A – Assignment 1 Submission

REQUIREMENTS SPECIFICATION

Favourite Books Online Store

WILLIAM REEDIE, NGY KEANG OEUNG, VAN TUY, ANH PHAN
104300597, 104201881, 104482107, 104480541

Contents

1. Introduction

Favourite Books is a bookstore located on Glenferrie Road. They have identified the popularity of online bookstores and wish to branch out into this area to increase their business reach.

This specification document will describe an online bookstore system that will support the operations of Favourite Books, including book browsing, purchasing, and management of orders and inventory.

The purpose of this document is to outline the requirements and quality attributes of the proposed online bookstore system for Favourite Books, which is intended to support the transition from a physical store to an online platform and reduce the limitations of the current manual processes. The functional requirements have been identified and organised into user tasks using the Tasks Description approach, focusing on what users need to perform within the system. An overview of the domain model is provided to show the relationships between key entities and actors. Quality attributes are included to ensure the system meets requirements such as security and usability. Finally, multiple possible solutions are considered to demonstrate how the system may be implemented.

2. Project Overview

2.1 Domain Vocabulary

Customer – Potential clients who browse the system for books.

Book – The product being sold.

Catalogue – A collection of books available for customers to view.

Cart – A temporary list where selected books are stored before checkout.

Order – Created when a customer confirms a purchase.

Payment – The process of completing payment for an order.

Invoice – A record showing the details and total cost of an order.

Shipment – The delivery of books to the customer after purchase.

2.2 Goals

Beyond just increasing the market reach of the store, the system also aims to centralise data regarding supply and sales statistics. To meet all these goals, the system must:

- Enable customers to browse the book catalogue online
- Support account creation and management for customers
- Provide shopping cart and order placement functionality
- Create invoices and receipts for customers
- Manage packing and shipment logistics
- Provide daily, weekly, monthly, and yearly sales statistics
- Allow the bookstore to update the catalogue

2.3 Assumptions

It can be assumed that customers have access to the internet and basic computer literacy skills, that the store has the necessary inventory and resources to support sales and delivery, and that many existing customers will be creating accounts to use the new online system.

As the system relies upon online transactions, it is assumed that payments will be handled through a secure external service. If payments are not managed properly, this may result in failed transactions or security risks, which could affect both customers and the business. For this reason, the orders will only be confirmed after payment has been successfully completed.

It can also be assumed that proper security measures are in place to protect customer data and transactions from unauthorised access or potential breaches, and access will be restricted so that only the owner/administrator has permission to update and manage the system, while staff are limited to handling packaging and order preparation.

2.4 Scope

The project cannot be expected to draw from a large budget, as it is being developed for a small store. The service must operate securely as to protect customer data and payment information. For the time being, discounts, loyalty programs, and rent-before-buying systems are not to be implemented, but may be considered in the future, and as such the system should be developed in a way to flexibly facilitate such an expansion of scope.

3. Domain Analysis

3.1 Pain Points

- Limited Accessibility
  - Customers are required to visit the physical store to browse and purchase books
  - This restricts convenience and reduces overall reach
- Inefficient Processes
  - Orders and stock are handled manually
  - As the business grows, this can lead to delays and *increase the chance of mistakes*
- Data Management Issues
  - Information is not stored in a central system
  - Difficulty in keeping records consistent and easily accessible
- Lack of Insights
  - No clear way to monitor sales or performance over time
- Operational Limitations
  - System access is limited
  - Difficulty for owners/staff to manage tasks at the same time

3.2 Domain Entities:

- Book
- Catalogue
- Order
- Order Item
- Customer
- Payment
- Invoice and Receipt
- Shipment
- Category
- Cart
- Inventory/Stock
- Sales Report

3.3 Actors

- Customer
- Administrator/Owner
- Employee
- Payment Service Provider(s)

3.4 Tasks List

Task 1: Create Customer Account

Task 2: Sign in to System

Task 3: Search and Browse Books

Task 4: View Books Details

Task 5: Add Books to Cart

Task 6: Modify Shopping Cart

Task 7: Review and Confirm Order

Task 8: Complete Payment Process

Task 9: Generate Invoice and Receipt

Task 10: Check Order Process

Task 11: Manage Book Listing and Classification

Task 12: Order Packaging and Delivery

Task 13: Analyse Sales Performance

4. Task Descriptions

Task 1: Create Customer Account

Task: 1	**Register Customer Account**
Purpose:	Register a new account
Trigger/Precondition	Customer does not have existing account
Frequency:	Once for each new customer registering an account
Critical:	Customer enters an existing account email or invalid details
Work Area:	Website
Sub-tasks:	
1. Customer inputs details	Registration form
2. Validate input	Check that required fields have been filled such as email, name, password etc
3. Verify input	Verify email
4. Create account	Store details securely in the system database
5. Confirm account creation	Prompt confirmation of successful registration
Variants:	
2a. Missing essential details	Highlight required field
3a. Existing customer account	Prompt account status (as existing customer)

Task 2: Sign in to System

Task: 2	Account Sign In
Purpose:	Access to an existing account
Trigger/Precondition	Customer has an existing account
Frequency:	Low, usually once per session
Critical:	Invalid username or password
Work Area:	
Sub-tasks:	
1. Enter username and password	Prompt user for login details
2. Validate credentials	Scan the database for matching credentials
3. Grant Access	Start session
4. Redirect to main page	System loads the user interface
Variants:	

Task 3: Search and Browse Books

Task: 3	Browse Book Catalogue
Purpose:	Allow customers to locate books within the catalogue
Trigger/Precondition	Customer accesses the website
Frequency:	High
Critical:	Handling large volumes of data efficiently
Work Area:	
Sub-tasks:	
1. View available books	System displays a list of books in the catalogue
2. Apply search or filter options	System provides search and filtering features
3. Display matching results	System shows results based on customer input
4. Select a book	System opens the selected book details
Variants:	
3a. No matching results	System displays a message indicating no results found

Task 4: View Book Details

Task: 4	View Book Details
Purpose:	Customer to view information
Trigger/Precondition	Customer clicks or selects a book
Frequency:	High
Critical:	Missing or incorrect book details
Work Area:	Book description page
Sub-tasks:	
1. Open the selected book	Load the book page
2. Show book details (title, price, author)	Information is displayed clearly
3. Show stock status	Display available stock
4. Choose next action (add to cart or return to main page)	Customer can add or exit the page
Variants:	
3a. Book is out of stock	Display a message to inform the user, and option to notify when available

Task 5: Add Books to Cart

Task: 5	Add Book to Cart
Purpose:	Allow the customer to add a book into the cart for buying
Trigger/Precondition	Customer is viewing a book
Frequency:	High
Critical:	Book not available
Work Area:	
Sub-tasks:	
1. Choose quantities	Customer enters quantity
2. Add book into cart	Put the book into cart
3. Update total prices	Update the cart total
4. Show confirmation	Prompt message confirming book added to card
Variants:	
1a. Book out of stock	Add book to cart action is disabled and shows a notice

Task 6: Modify Shopping Cart

Task: 6	Update Shopping Cart
Purpose:	Allow user to review and modify their cart before purchase
Trigger/Precondition	Shopping cart contains one or more items
Frequency:	Low, usually only a few times per session
Critical:	Incorrect total price calculation
Work Area:	
Sub-tasks:	
1. View cart items	Display all items currently in the cart
2. Change quantities	Update the quantity of selected items
3. Remove books	Remove the selected book from the cart
4. Recalculate total prices	Update the total price accordingly
Variants:	
1a. Shopping cart is empty	Prompt error message to notify user

Task 7: Review and Confirm Order

Task: 7	Checkout Order
Purpose:	Allow customers to confirm and place their order
Trigger/Precondition	Shopping cart contains one or more items
Frequency:	When customer proceeds to place an order
Critical:	Missing or incomplete required information
Work Area:	
Sub-tasks:	
Enter delivery details	Customer inputs the address form
Review order summary	System displays all selected books and total cost
Confirm order details	Customer verifies details and proceeds to checkout
Variants:	
Required fields missing	System prompts an error message and prevents checkout from continuing

Task 8: Complete Payment Process

Task: 8	Process Payment
Purpose:	Verify payment and complete purchase
Trigger/Precondition	Check out process is completed
Frequency:	Low, around once per session or less
Critical:	Payment transaction fails
Work Area:	
Sub-tasks:	
1. Enter payment details	Customer inputs payment information
2. Verify payment	Send the request to a third-party payment service and will confirm whether payment is successful or failed
3. Record transaction	System updates and stores payment details in the system
4. Update stock level	System updates the available quantity after successful payment
Variants:	
2a. Payment failure	System displays an error message for a failed payment and allows customer to retry with a different card/payment method

Task 9: Generate Invoice and Receipt

Task: 9	Generate Invoice and Receipt
Purpose:	Provide user with proof of purchase
Trigger/Precondition	Payment is successfully completed
Frequency:	After each successful payment
Critical:	Incomplete or incorrect transaction records
Work Area:	
Sub-tasks:	
1. Generate invoice	System creates an invoice record for the order
2. Generate receipt	System creates a receipt for the completed payment
3. Store records	System saves the transaction data in the database
4. Display and send receipt	Displays the receipt and sends a copy to customer's email
Variants:	

Task 10: Check Order Process

Task: 10	Track Order Status
Purpose:	Customers can check the delivery progress of their order
Trigger/Precondition	An existing order is available
Frequency:	Low
Critical:	Outdated or incorrect status
Work Area:	
Sub-tasks:	
1. View order history	System displays all user orders
2. Select an order	Customer chooses a specific order to view the status
3. View order status	System shows the current status and ETA (e.g. processing, shipped, delivered)
Variants:	

Task 11: Manage Book Listing and Classification

Task: 11	Manage Book Catalogue and Categories
Purpose:	Allows administrators to manage and update book catalogue and categories
Trigger/Precondition	Administrator has logged into the system
Frequency:	When administrators update or maintain the catalogue
Critical:	Inconsistency or incorrect information
Work Area:	
Sub-tasks:	
1. Add or edit book details	Admin inputs or updates book information or stock
2. Assign book category	System connects the book to the selected category
3. Monitor stock level	System displays stock quantity and highlights low stock items
4. Save changes	System stores and updates the changes
5. Display changes	System reflects changes
Variants:	

Task 12: Order Packaging and Delivery

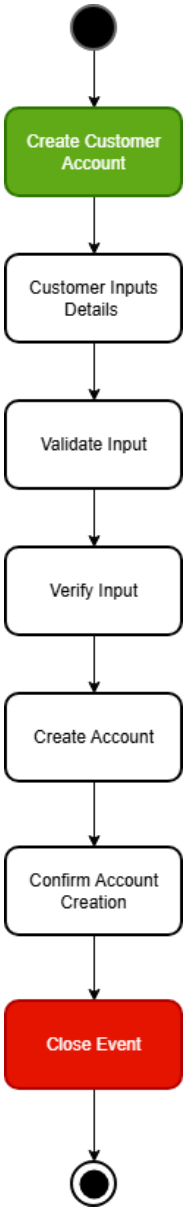
Task: 12	Process Packaging and Shipment
Purpose:	Ensure orders are prepared and delivered to customers
Trigger/Precondition	Order has been successfully paid
Frequency:	After an order is processed for delivery
Critical:	Incorrect delivery or shipment details
Work Area:	
Sub-tasks:	
1. View paid orders	System displays a list of successful purchase orders ready for processing
2. Package items	Staff prepare and pack the selected books
3. Create shipment record	System generates a shipment entry for the order
4. Update delivery status	System marks the order as shipped, provides tracking number and updates ETA
Variants:	

Task 13: Analyse Sales Performance

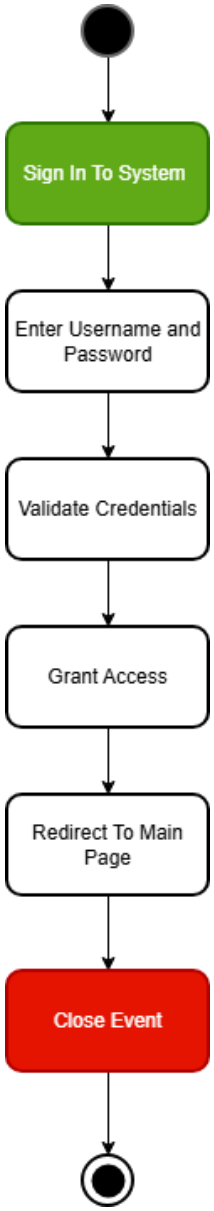
Task: 13	View Sales Statistics
Purpose:	Allow administrators to monitor sales performance
Trigger/Precondition	Sales data is available in the system
Frequency:	Daily when administrators review performance
Critical:	Incomplete data
Work Area:	
Sub-tasks:	
1. Select time period	Admin chooses a specific timeframe (e.g. daily, monthly)
2. Retrieve sales data	System collects relevant sales information
3. Display sales summary	System shows total sales and trends
4. Identify best selling books	System highlights best selling books or categories
Variants:	

5. Workflow Diagrams

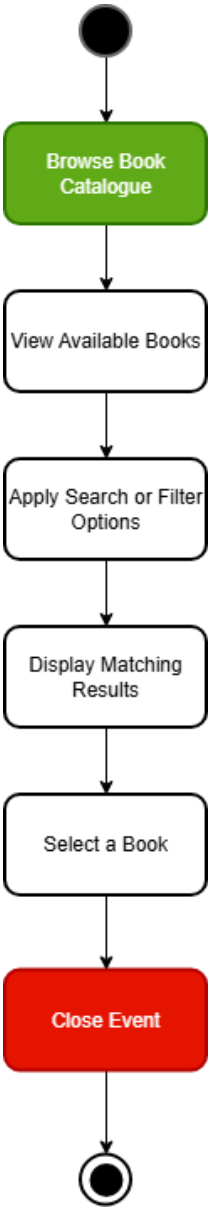
Task 1: Create Customer Account



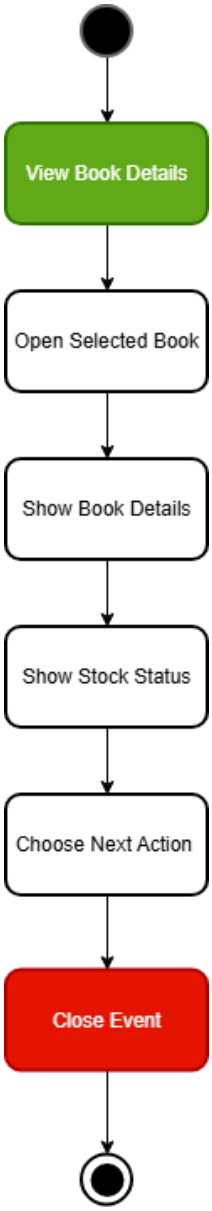
Task 2: Sign in to System



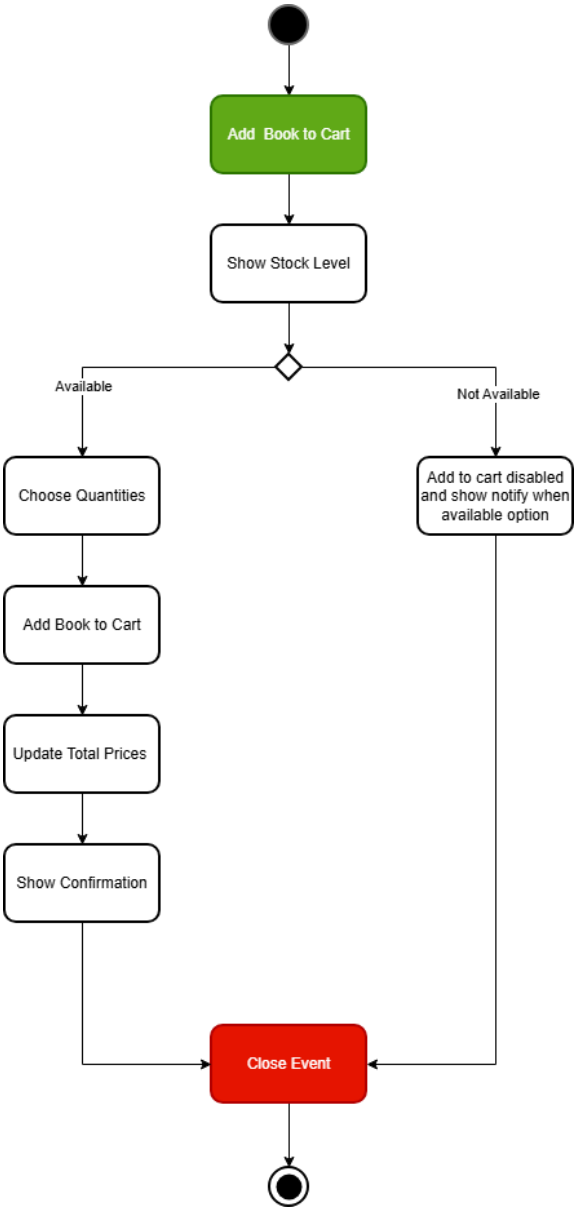
Task 3: Search and Browse Books



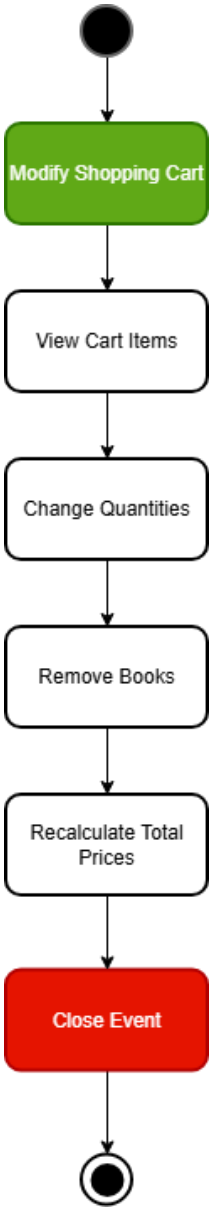
Task 4: View Books Details



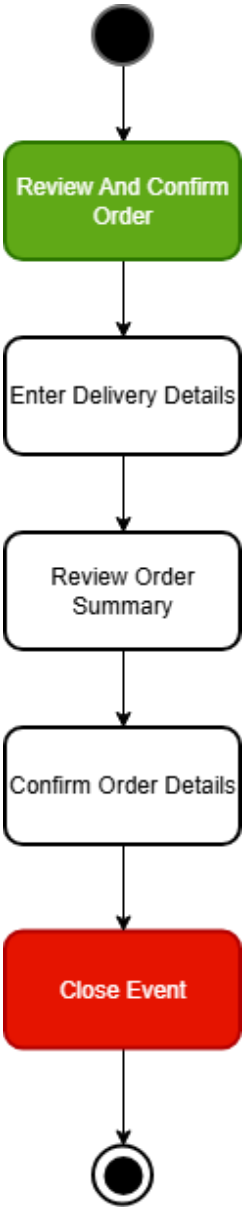
Task 5: Add Books to Cart



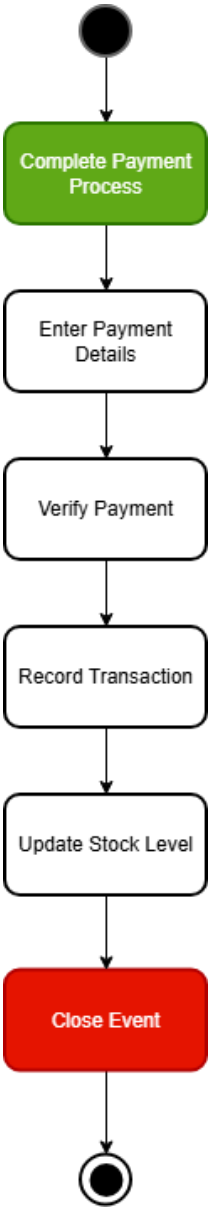
Task 6: Modify Shopping Cart



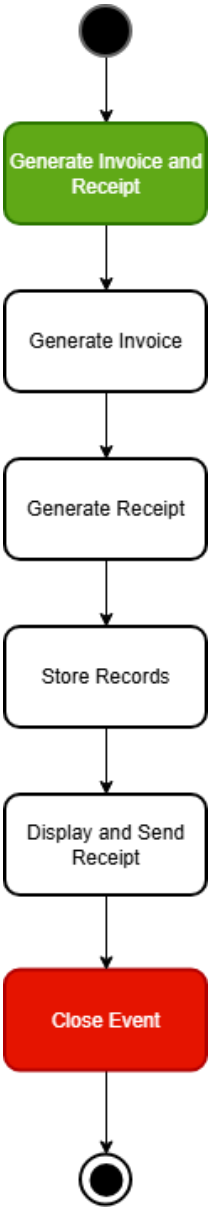
Task 7: Review and Confirm Order



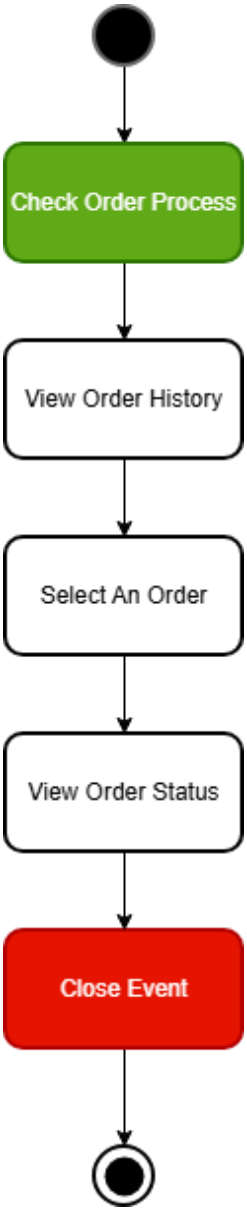
Task 8: Complete Payment Process



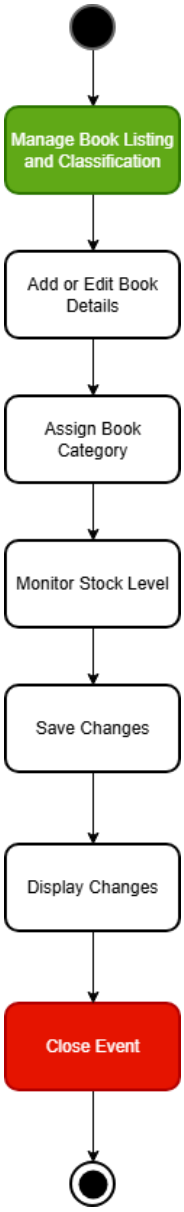
Task 9: Generate Invoice and Receipt



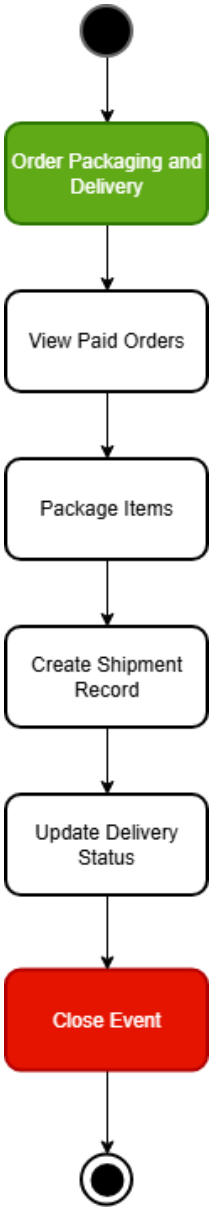
Task 10: Check Order Process



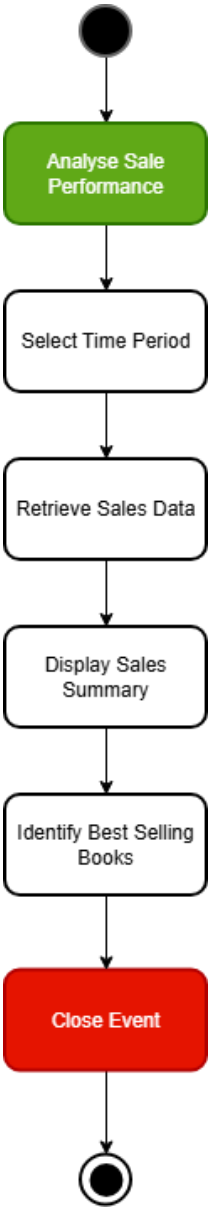
Task 11: Manage Book Listing and Classification



Task 12: Order Packaging and Delivery



Task 13: Analyse Sale Performance



6. Product Design

6.1 Data / Domain Model

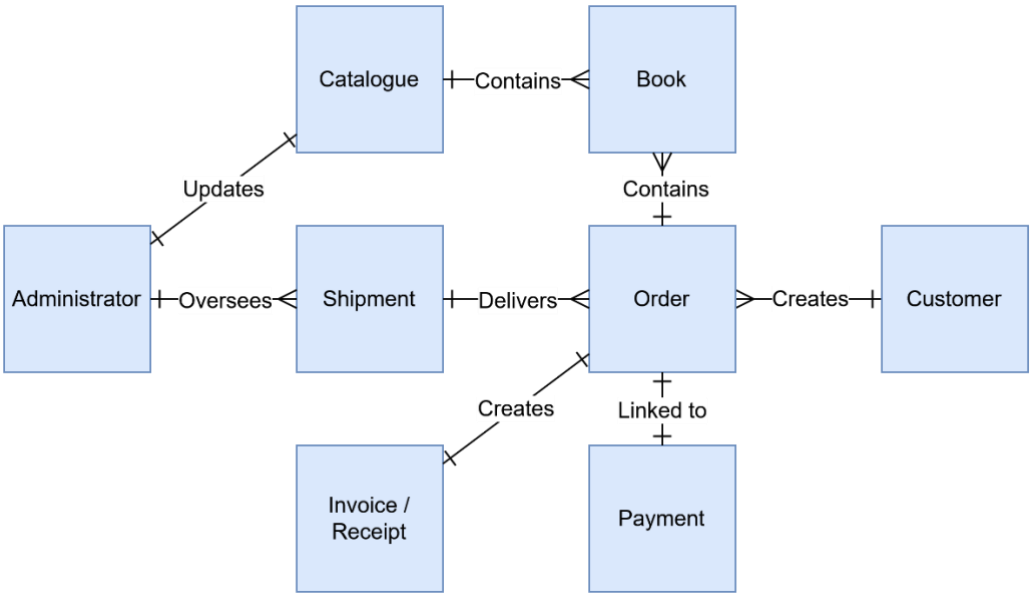


Figure 1 Domain Model of the System

6.2 Functional Requirements

The Favourite Books Online Store must provide the functions needed to support both customer-facing online purchasing and internal bookstore operations.

At the customer level, the system shall allow users to:

- create an account
- sign in securely
- browse and search the book catalogue
- view detailed book information
- add books to a shopping cart
- modify cart contents
- review and confirm orders
- complete payment
- receive an invoice and receipt
- check the progress of their orders

At the administrative level, the system shall allow bookstore staff to:

- manage book listings and classification
- update the catalogue as new books become available
- process packaging and shipment of completed orders
- analyse sales performance over selected time periods
- view sales statistics on a daily, weekly, monthly, yearly, and year-to-date basis

The system shall enforce basic operational integrity by:

- validating customer registration details
- ensuring that payment is confirmed before an order is treated as complete
- updating stock levels after a successful purchase
- maintaining accurate transaction records for invoices and receipts
- preventing checkout when required information is incomplete
- handling failed payment attempts by notifying the customer and allowing another attempt

6.3 Quality Attributes

Security

Security is critical because the system stores sensitive customer information such as account details, order history, and payment related to important data. If this information is compromised it may lead to financial loss, privacy breaches and damage to the business reputation. Therefore, strong protection are required to ensure that only authorised users can access the system and that all transactions are handled securely.

To meet this requirement, the system shall:

- require secure authentication for both customers and administrative users
- restrict access to system functions based on user roles and permissions
- securely store sensitive data such as passwords using encryption
- prevent unauthorised access to administrative features such as catalogue management and reporting
- protect against common security risks such as invalid login attempts and unauthorised data access

Usability

Usability is important because the system is designed for general bookstore customers and staff with varying levels of technical experience. A system that is difficult to use may lead to user frustration, errors during checkout and reduced sales. The system must therefore provide a simple and clear interface that supports users in completing tasks efficiently.

To meet this requirement, the system shall:

- provide clear and navigation for browsing, searching and purchasing books
- allow users to easily manage their shopping cart and review orders before checkout
- display clear and helpful error messages for invalid inputs, failed payments or missing information
- support efficient completion of common tasks such as account registration, ordering and tracking deliveries
- follow existing user interface guidelines to ensure a familiar and standardised user experience

Performance

Performance is important because the system will be frequently used for browsing books, searching for items and completing purchases, so slow response times may negatively impact user experience and lead to abandoned transactions. The system must therefore provide fast and responsive performance under normal usage conditions.

To meet this requirement, the system shall:

- return search and browsing results within an acceptable response time under normal conditions
- process shopping cart updates and checkout operations without noticeable delay
- support multiple users accessing the system simultaneously without significant performance degradation
- generate administrative reports efficiently for selected time periods
- ensure that system response remains stable during peak usage periods

Reliability

Reliability is important because customers must be able to trust that orders, payments, invoices and delivery updates are recorded correctly. Any errors may lead to incorrect transactions, delays or customer dissatisfaction.

To meet this requirement, the system shall:

- preserve transaction accuracy for orders, payments and invoices
- prevent duplicate, incomplete or lost orders
- ensure that orders are only marked as completed after successful payment confirmation
- maintain accurate and up to date order status information
- update stock and shipment data consistently after each transaction
- handle failures such as payment errors without losing data

6.4 Other Requirements

Product Level Requirements

The Online Bookstore System shall be provided as a web-based software product that supports customer access through standard internet-connected devices and supports internal staff operations such as order processing and reporting.

To meet this requirement, the system shall:

- support core bookstore functions including catalogue browsing, ordering, payment handling, invoice and receipt generation, shipment support, and sales reporting
- limit the system scope to core bookstore operations only
- exclude features such as discounts, loyalty programs, and rent-before-buying in the current version
- allow future extensions without restricting the addition of new features
- integrate with an external payment service for secure transaction processing
- support integration with external delivery services for shipment handling
- operate as a software solution only, with hardware and deployment handled separately

Design Level Requirements

The system shall be designed in a modular structure so that key functions such as account management, browsing, shopping cart, order processing, payment, shipment and reporting are separated into distinct components.

To meet this requirement, the system shall:

- separate major system functions into independent modules to improve maintainability
- support future system expansion without requiring major redesign
- maintain consistency across customer and administrative interfaces
- enforce appropriate access control between customer and staff functions

- follow existing organisation interface guildelines for user interface design

7. Validation and Verifiability

Validation and verifiability are essential to ensure that the system meets user needs and that all requirements can be tested effectively. In this project, validation ensures that the Favourite Books Online Store is the correct solution by reviewing requirements against real-world bookstore operations, aligning them with stakeholder needs such as online sales, inventory management, and delivery tracking, and using scenario-based evaluation (e.g. browsing books, adding items to a cart, and completing payment) to confirm that all user workflows are fully supported without missing steps. Consistency checks are also applied to ensure there are no conflicts between requirements, such as maintaining correct stock updates after successful payments and accurate order tracking during delivery. Verifiability ensures that each requirement can be objectively tested through functional testing of system tasks (e.g. account creation, login, cart management, and payment processing), input and output validation using both valid and invalid data, and system behaviour testing under critical conditions such as payment failures or out-of-stock items. In addition, data integrity checks confirm that transactions, invoices, and stock levels are correctly recorded and updated, while performance verification ensures that key operations such as searching the catalogue and completing checkout occur within acceptable response times. Together, these validation and verification approaches ensure that the system is both correct in design and reliable in operation.

CRUD Check

Entity / Task	Customer	Book	Catalogue	Order	Payment	Invoice/ Receipt	Shipment
Create Customer Account	C						
Sign in to System	R						
Search and Browse Books		R	R				
View Book Details		R					
Add Book to Cart		R					
Modify Shopping Cart		R					
Review and Confirm Order	R	R		C			
Complete Payment Process	R			R, U	C		

Generate Invoice and Receipt	R			R	R	C	
Check Order Process	R			R			R
Manage Book Listing and Classification		C, R, U, D	C, R, U, D				
Analyse Sales Performance				R	R	R	
Order Packaging and Delivery				R, U		R	C, U

8. Solutions

8.1 Possible Solution: Website-Based Online System

This solution aims to develop a website-based online bookstore system that would help the business expand its reach from a local store to across Australia. Currently, the majority of customers come from the surrounding area, with all transactions being done in person. Records are handled on paper without a proper managing system, which is suboptimal, reducing management efficiency. Therefore, by introducing a website, it would allow the business to extend its reach, improve customer experience through digitisation, and improve daily operations. The system would aim to improve accessibility with reliability, as customers can now place orders digitally. On top of this, the business would now have records of past orders and analytics to support management goals.

Once the system has been setup, the owners would be able to manage their inventory, update new arrival books, or update them to match their current stock level. This information is required to be up to date as incorrect display details could lead to failed orders and customer dissatisfaction. Furthermore, the website would allow customers to search, browse and view book details before ordering. This allows an easy and convenient purchasing process for customers.

To make a purchase, customers would be required to log in to their account, and new users would need to register beforehand. By doing this, order information and customers details can be recorded and securely stored in the system for analysis and tracking. Additional functions would also be provided such as adding books to cart, reviewing and editing shopping cart before proceeding to checkout. This additional stage would minimise errors and give customers the flexibility to review and adjust if needed. Following successful completion of the order through an external service, a confirmation of order would be presented to customer, and a copy of receipt would be sent to their email address. Due to the sensitive nature of online transactions, a secured handling is required to reduce risks such as unauthorised access or failed payments. This means, orders can only be processed after payment is successfully confirmed, and an invoice/receipt is generated and stored as a record of the transaction.

After successful payment has been confirmed, stock levels will be updated automatically to accurately reflect the available stock. Failure to update this could lead to overselling or delays in fulfilment of orders. Store staff could then proceed with packaging and preparing for shipment, with shipment details recorded in the system to track delivery progress. This would allow for transparency by allowing customers to sign into their accounts and track the progress of the order without having to raise an enquiry to the store directly.

Aside from supporting the customer shopping experience, the system would also support the business in managing their operations. The owners could login as administrators through the same platform to manage categories and update their stock records. This system aims to provides sales information over different time periods, which would provide the business the ability to review their performance and supports in decision making. This solution helps Favourite Books reach more customers, makes daily operations easier to manage, and supports the business as it continues to grow.

8.2 Possible Solution: Mobile app based solution

The development of a mobile based online bookstore system, which would be designed to offer a more accessible and customized user experience than a standard website, is a potential alternate solution. Customers would be able to easily browse books, search for specific titles, examine full details, and make purchases from their devices at any time with a mobile app thanks to the dominant popularity of smartphones. The app would offer key characteristics including account registration, safe login, shopping cart management, checkout, and payment processing through an outside payment service to guarantee transaction security, just as the online system.

Additionally, a mobile application could increase customer engagement and retention by providing improved features like push notifications to let users know about new arrivals, specials, or order status updates. Additionally, the system would keep data synchronized in real-time, guaranteeing that order details, transaction records, and stock levels would be regularly updated. On an administrative level, the backend system would be unchanged, enabling employees to effectively handle inventories, handle deliveries, and evaluate sales results. However, platform-specific factors would include compatibility with iOS and Android, and higher early development and maintenance expenses might be necessary when creating a mobile application. Despite these difficulties, the mobile app solution would offer better user interaction, increased convenience, and long-term scalability, making it a viable option for boosting client satisfaction and promoting business growth.